

Supply Chain Performance & Forecast Analytics

SQL-Based Supply Chain Performance Analysis with KPI Monitoring and Forecast Evaluation



Prepared by: Aylin Öztürk

Course: Semester Project

Instructor: Prof. Dr. Beytullah Gültekin Çetiner

Department: Management Information Systems and Engineering

Marmara University

May 2026

ABSTRACT

This project aims to design and implement a relational database system for analyzing supply chain performance using SQL-based analytical techniques. A simulated supply chain dataset was created and processed through an Extract, Transform, and Load (ETL) workflow before being imported into a MySQL database. The database structure was designed using normalization principles and supported by an Entity-Relationship (ER) model to ensure data integrity and consistency.

Several key performance indicators (KPIs), including On-Time Delivery Rate, OTIF (On-Time In Full), Forecast Accuracy, Inventory Turnover, and Stockout Rate, were calculated to evaluate operational performance. Advanced SQL queries, views, indexing techniques, and analytical functions were utilized to identify supplier performance issues, forecast deviations, inventory risks, and demand trends. In addition, an interactive Power BI dashboard was developed to visualize performance metrics and support managerial decision-making.

To enhance usability, a web-based front-end prototype was implemented using Flask, allowing users to enter and manage supply chain data through a simple interface. The results demonstrate how database technologies and analytical methods can transform operational data into actionable business insights. The project highlights the importance of data-driven decision-making in improving supplier reliability, forecasting accuracy, and overall supply chain efficiency.

Keywords: Supply Chain Management, SQL Analytics, Forecast Accuracy, KPI Monitoring, Inventory Management, Power BI, Database Systems, Decision Support.

1. Introduction	4
2. Project Objective	4
3. Business Scenario	5
4. Data Model (ER Design)	5
5. Database Schema	6
6. Data Preparation & ETL	6
7. Data Quality Management.....	8
7.1. Missing Value Check.....	8
7.2. Duplicate Record Check.....	9
7.3. Negative Value Check	9
7.4. Date Validation	10
7.5. Referential Integrity Check.....	10
8. KPI Framework.....	11
8.1. On-Time Delivery Rate.....	11
8.2. OTIF (On Time In Full).....	12
8.3. Forecast Accuracy	13
8.4. Inventory Turnover.....	14
8.5. Stockout Rate	15
9. SQL Analysis.....	16
9.1. Supplier performance + delay analysis	16
9.2. Demand trend analysis	18
9.3. Forecast deviation analysis.....	20
9.4. Window function	21
10. Views & Advanced SQL	23
10.1. Supplier Performance View	23
10.2. Forecast Accuracy View.....	24
11. Database Optimization	26
12. Normalization	27
13. Business Insights & Recommendations	27
13.1. Business Insights	28
13.1.1. Supplier Performance Issues	28
13.1.2. Increasing Demand Trend.....	28
13.1.3. Forecast Accuracy Problems.....	28

13.1.4. Inventory Management Imbalance	28
13.1.5. Uneven Risk Distribution	28
13.2. Recommendations.....	28
13.2.1. Improve Supplier Selection and Monitoring.....	28
13.2.2. Enhance Forecasting Models	29
13.2.3. Implement Product-Level Inventory Strategies.....	29
13.2.4. Align Inventory with Demand Trends	29
13.2.5. Reduce Excess Inventory.....	29
14. Power BI Dashboard	29
14.1. On-Time Delivery Rate	29
14.2. Stockout Rate.....	29
14.3. Inventory Turnover	30
14.4. Forecast Accuracy	30
15. Analytical Thinking Approach.....	30
16. Limitations & Future Work.....	31
17. Final Deliverables.....	31
18. Supply Chain Management Front-End Prototype.....	31
18.1. Main Dashboard Screen.....	33
18.2. Supplier Entry	33
18.3. Product Entry	34
18.4. Purchase Order Entry	34
18.5. Forecast Entry.....	35
19. Conclusion	36
20. References.....	37

Supply Chain Performance & Forecast Analytics

1. Introduction

Supply chain management has become increasingly important in modern business environments due to the growing complexity of global operations and customer expectations. Organizations generate large volumes of operational data related to suppliers, inventory, procurement activities, and customer demand. However, transforming this data into meaningful information requires effective database systems and analytical tools.

Data-driven decision-making plays a critical role in improving supply chain performance. Companies rely on key performance indicators (KPIs), forecasting techniques, and performance monitoring systems to identify operational inefficiencies and support strategic planning. Database technologies such as SQL provide powerful capabilities for storing, managing, and analyzing large datasets, enabling organizations to gain valuable business insights.

This project focuses on the design and implementation of a supply chain analytics system using MySQL, SQL queries, Power BI, and a Flask-based front-end application. A relational database was developed to manage supply chain data, including suppliers, products, inventory levels, purchase orders, forecasts, and actual demand records. Various analytical techniques were applied to evaluate supplier performance, demand trends, forecast accuracy, and inventory risks.

The main objective of the study is to demonstrate how database systems and analytical tools can be integrated to support supply chain decision-making. Through KPI calculations, SQL analysis, dashboard visualization, and front-end development, the project provides a practical example of transforming operational data into actionable business insights.

2. Project Objective

This project aims to design and implement a relational database system to analyze supply chain performance using SQL-based analytical techniques. The primary focus is to evaluate key operational dimensions such as supplier reliability, delivery performance, and demand forecast accuracy.

Through the integration of structured data modeling and analytical querying, the project seeks to transform raw operational data into meaningful insights. These insights will support data-driven decision-making processes and highlight critical inefficiencies within the supply chain.

In addition, the project will demonstrate the practical application of database management concepts, including normalization, data integrity, and query optimization, while also strengthening analytical thinking skills aligned with business and data analyst roles.

3. Business Scenario

In today's competitive business environment, supply chain efficiency plays a crucial role in ensuring customer satisfaction and operational success. This project is based on a simulated logistics company that is experiencing several operational challenges, particularly in terms of delivery delays and inaccurate demand forecasting.

These issues have led to increased operational costs, reduced service levels, and inefficiencies in inventory management. Late deliveries negatively impact customer satisfaction, while poor forecast accuracy results in either excess inventory or stockout situations.

The objective of this scenario is to identify the root causes behind these inefficiencies by analyzing structured supply chain data. The analysis will focus on supplier performance, delivery timelines, and the gap between forecasted and actual demand.

By leveraging SQL-based analysis and data modeling techniques, the project aims to uncover actionable insights that can improve supplier selection, enhance forecasting accuracy, and optimize overall supply chain operations.

4. Data Model (ER Design)

In this project, an Entity-Relationship (ER) data model was designed to represent the structure of a supply chain system. The model defines the main entities involved in supply chain operations and the relationships between them, providing a structured foundation for database implementation.

The key entities identified in the model include *Supplier*, *Product*, *Warehouse*, *Inventory*, *Purchase Order*, *Purchase Order Detail*, *Forecast*, and *Actual Demand*. Each entity represents a specific aspect of the supply chain, such as suppliers providing products, warehouses storing inventory, and demand reflecting actual sales or consumption.

Relationships between entities were established based on real-world business logic. For example, a *Supplier* can be associated with multiple *Purchase Orders*, and each *Purchase Order* can contain multiple *Purchase Order Details*. Similarly, *Inventory*, *Forecast*, and *Actual Demand* are linked to both *Product* and *Warehouse* entities, enabling multi-dimensional analysis.

The model was designed following normalization principles to reduce redundancy and ensure data consistency. Primary keys uniquely identify each entity, while foreign keys

define the relationships between them. This structure enables efficient querying, data integrity, and scalability.

The ER diagram was created using a data modeling tool, providing a clear visual representation of entities, attributes, and relationships, which was later used as the basis for database schema generation.

5. Database Schema

In this project, a relational database schema was designed and implemented to support supply chain operations and analytical processes. The database structure was developed using a data modeling approach, where entities such as *Supplier*, *Product*, *Warehouse*, *Inventory*, *Purchase Order*, *Forecast*, and *Actual Demand* were defined.

Each table was created with appropriate constraints to ensure data integrity and consistency. Primary keys (PK) were assigned to uniquely identify each record, while foreign keys (FK) were used to establish relationships between tables. These relationships enable the integration of transactional and master data, which is essential for analytical queries.

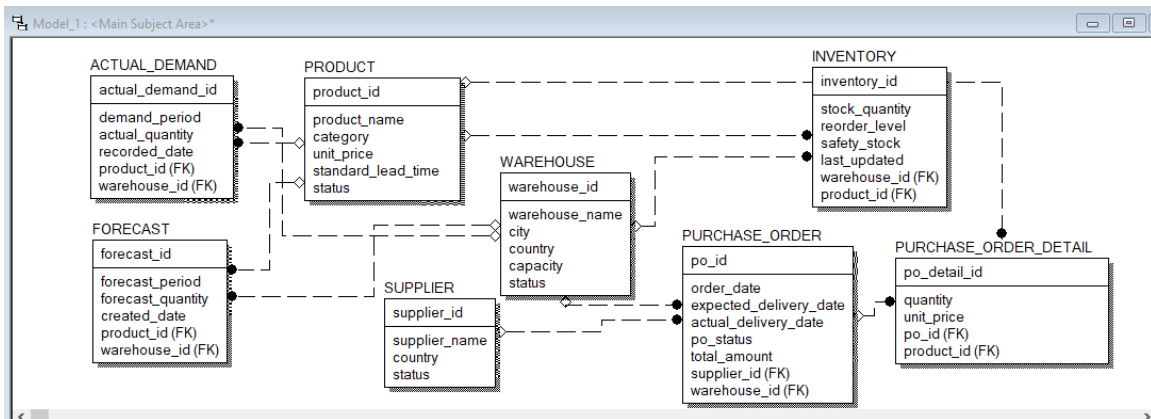


Figure 1. ER Diagram

The database schema was initially designed using a modeling tool and then converted into SQL scripts through a forward engineering process. These scripts were executed in MySQL to create the physical database structure.

6. Data Preparation & ETL

The dataset used in this project was synthetically generated in Microsoft Excel to simulate real-world supply chain operations. The data preparation process involved cleaning, transforming, and organizing the dataset to ensure compatibility with the database schema.

During the transformation phase, several adjustments were made:

- Date fields were standardized into the *yyyy-mm-dd* format to ensure proper handling in MySQL.
- Column names were aligned with the database schema to maintain consistency.
- Data types were reviewed to ensure correct mapping during import.

After the transformation process, each dataset was exported as CSV files. These files were then loaded into the MySQL database using the *Table Data Import Wizard*, which represents the Load phase of the ETL (Extract, Transform, Load) process.

This structured ETL approach ensured that the data was properly formatted, consistent, and ready for analytical processing within the database environment.

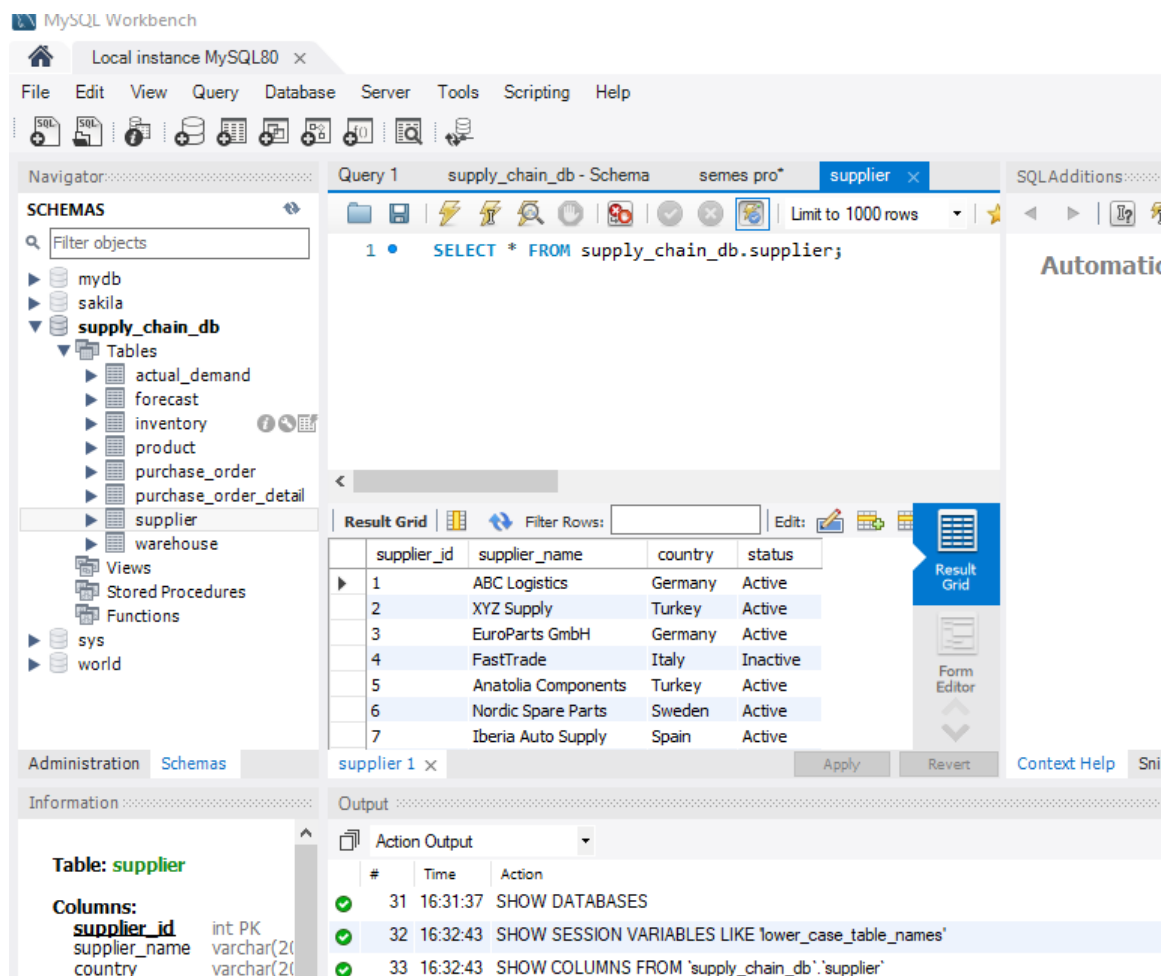


Figure 2. Data Import Process into MySQL Database

7. Data Quality Management

To ensure the reliability and accuracy of the analysis, data quality checks were performed on the dataset after it was loaded into the database. These checks aimed to identify and eliminate potential data issues such as missing values, duplicate records, invalid entries, and logical inconsistencies.

The following data quality validations were applied:

7.1. Missing Value Check

Tables were scanned for null or missing values in critical fields to ensure completeness of the dataset.

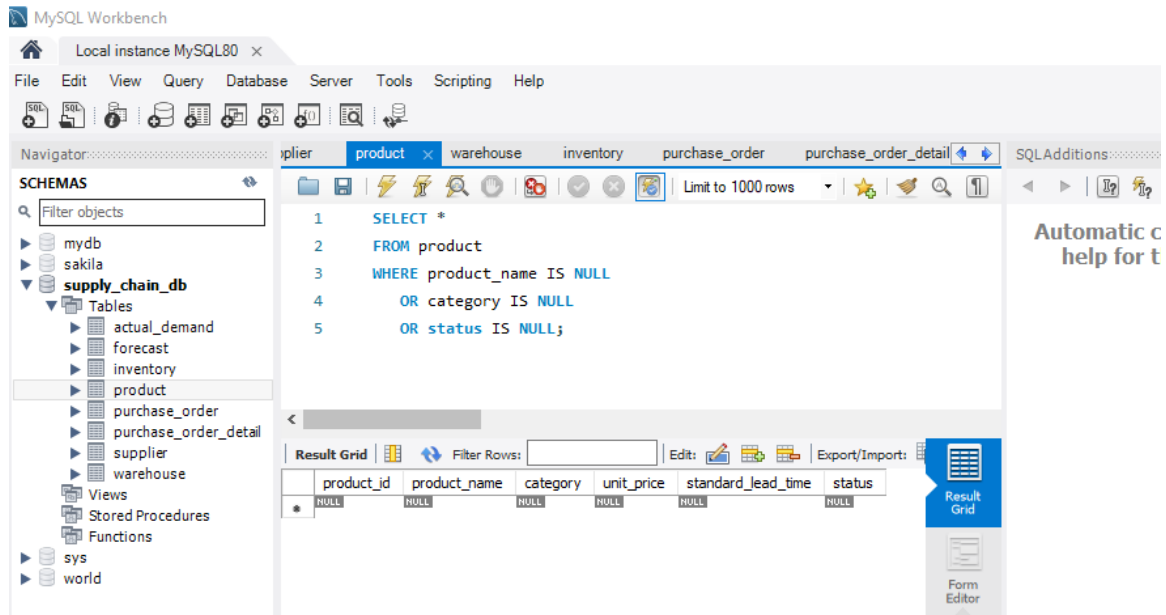


Figure 3. Missing Value Check

7.2. Duplicate Record Check

Primary key fields were analysed to confirm that each record is uniquely identified and no duplicates exist.

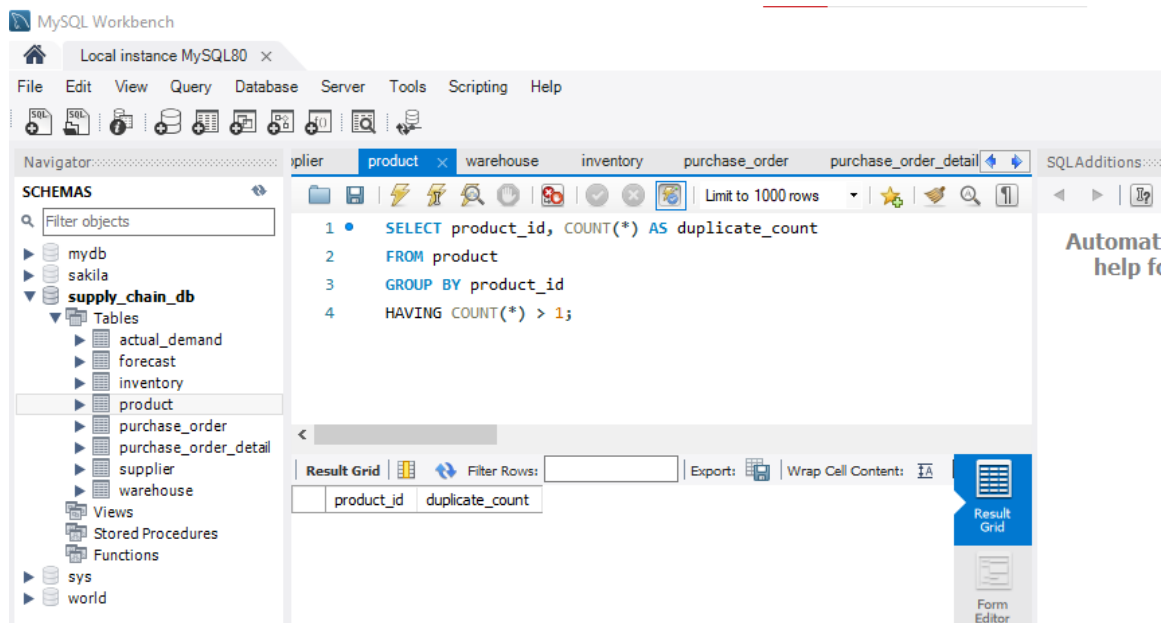


Figure 4. Duplicate Record Check

7.3. Negative Value Check

Numerical fields such as stock quantities and demand values were validated to ensure that no negative values were present.

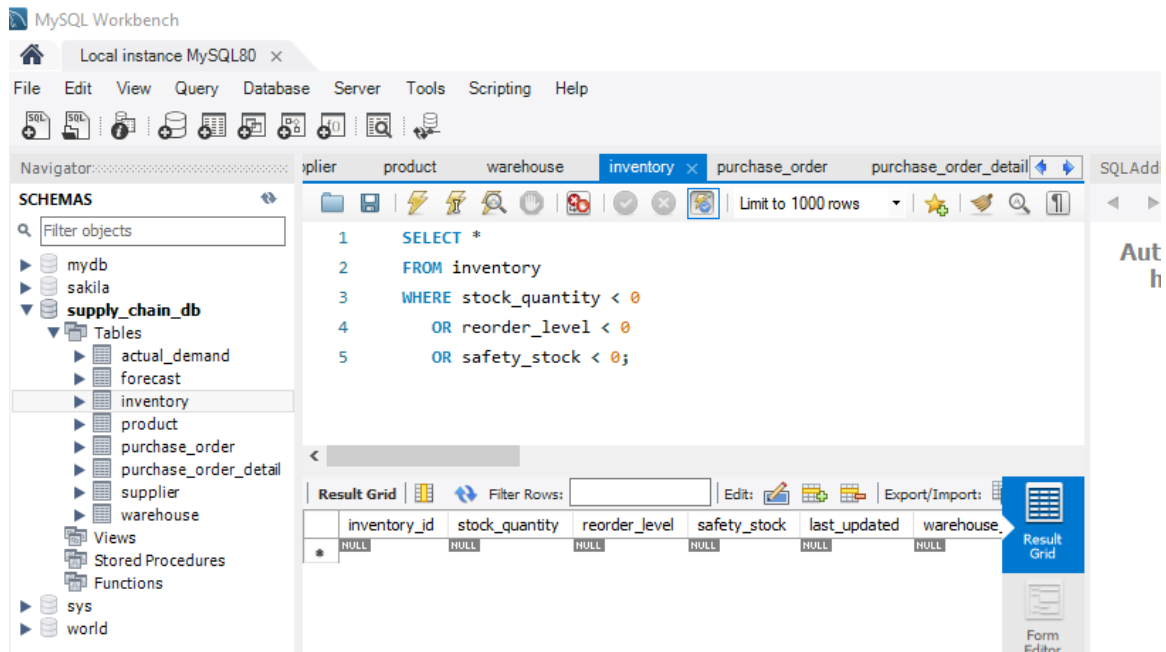


Figure 5. Negative Value Check

7.4. Date Validation

Date fields were checked for logical consistency (e.g., actual delivery dates should not be earlier than order dates).

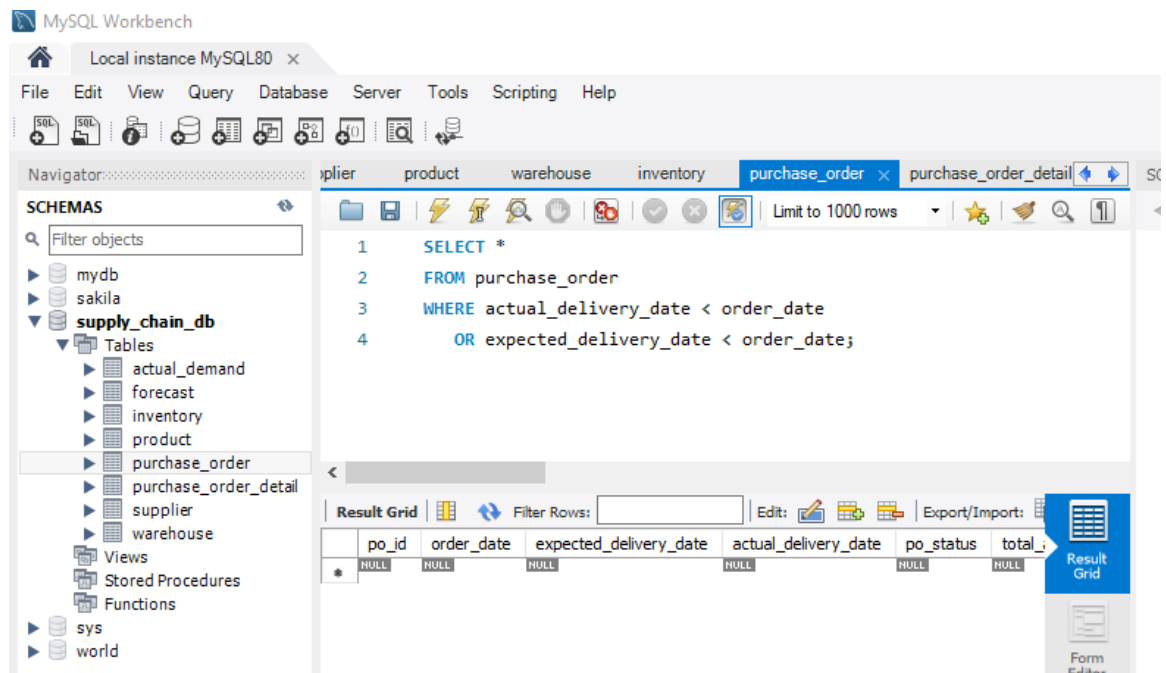


Figure 6. Data Validation

7.5. Referential Integrity Check

Foreign key relationships were validated to ensure that all references between tables are consistent and no orphan records exist.

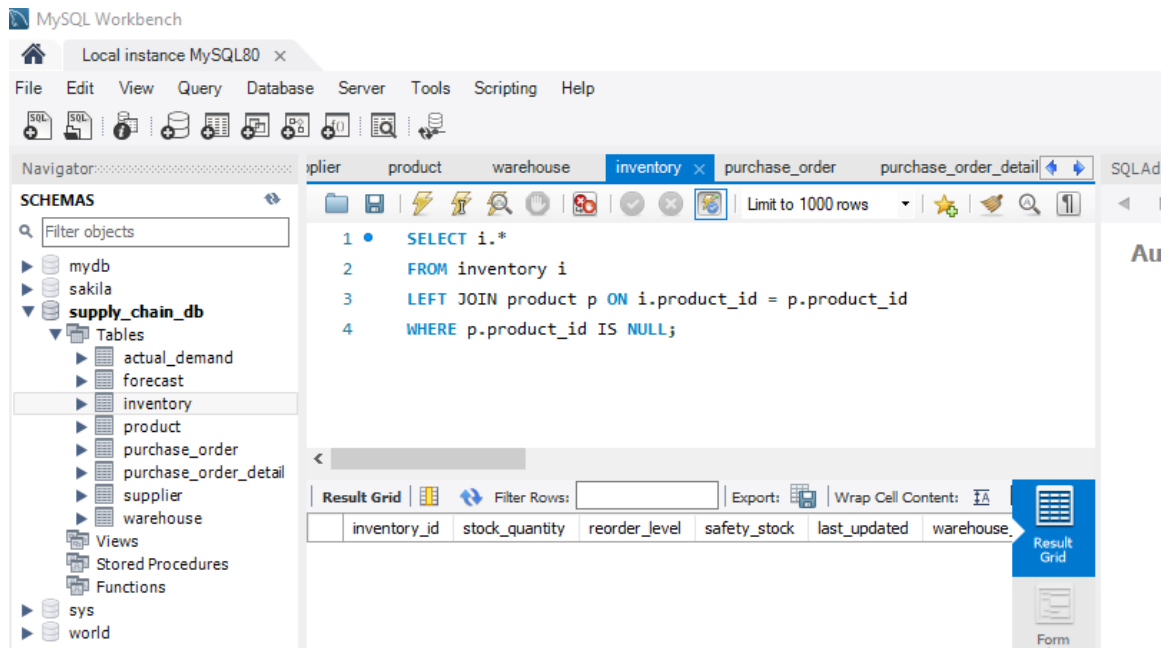


Figure 7. Referential Integrity Check

These validation steps were performed using SQL queries, and the results confirmed that the dataset is consistent, accurate, and suitable for further analytical processing.

8. KPI Framework

In order to evaluate supply chain performance, a set of key performance indicators (KPIs) was defined. These metrics provide insights into operational efficiency, service level, and planning accuracy.

8.1. On-Time Delivery Rate

This KPI measures the percentage of orders delivered on or before the expected delivery date.

On-Time Delivery Rate = (Number of On-Time Deliveries / Total Deliveries) × 100

```
SELECT (SUM(CASE
WHEN actual_delivery_date <= expected_delivery_date
THEN 1 ELSE 0 END) / COUNT(*)) * 100 AS on_time_delivery_rate
FROM purchase_order;
```

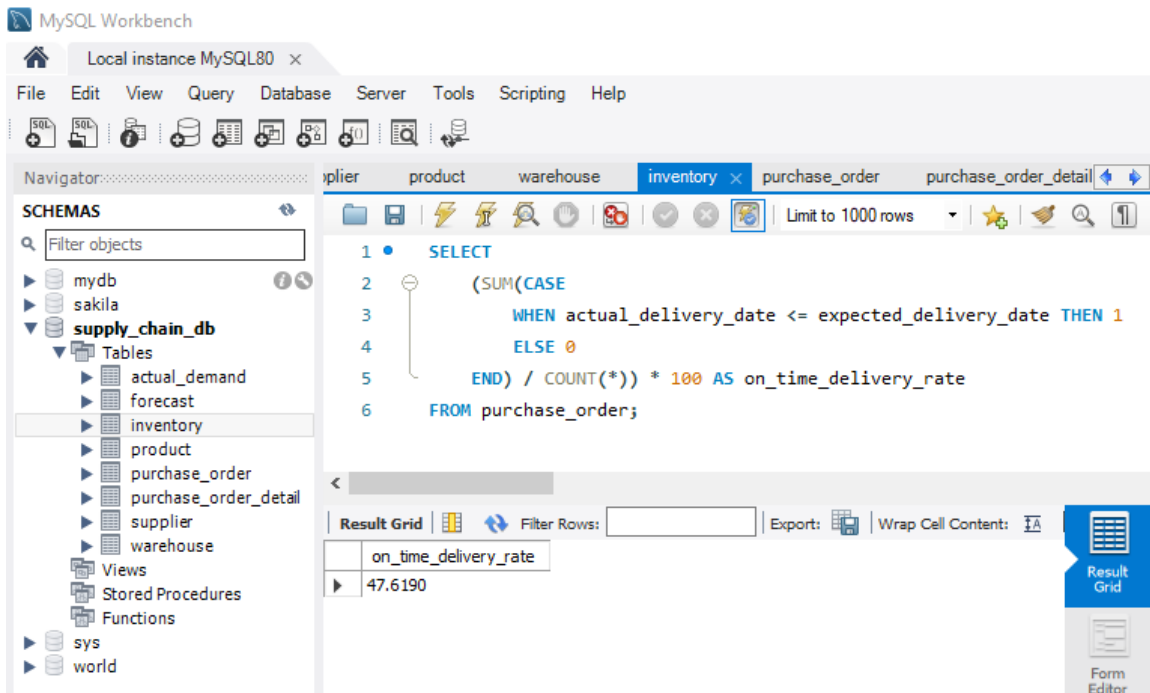


Figure 8. On-Time Delivery Rate

The On-Time Delivery Rate was calculated as 47.6%, indicating that less than half of the orders were delivered on time. This suggests potential inefficiencies in supplier performance or logistics operations.

8.2. OTIF (On Time In Full)

OTIF evaluates whether deliveries are completed both on time and in full quantity.

$$\text{OTIF} = (\text{Number of Orders Delivered On Time and In Full} / \text{Total Orders}) \times 100$$

```

SELECT
    ROUND(
        (SUM(CASE
            WHEN po.actual_delivery_date <= po.expected_delivery_date
                AND pod.quantity > 0
            THEN 1
            ELSE 0
        END) / COUNT(*)) * 100, 2
    ) AS otif_rate

```

```

FROM purchase_order po

JOIN purchase_order_detail pod

    ON po.po_id = pod.po_id;

```

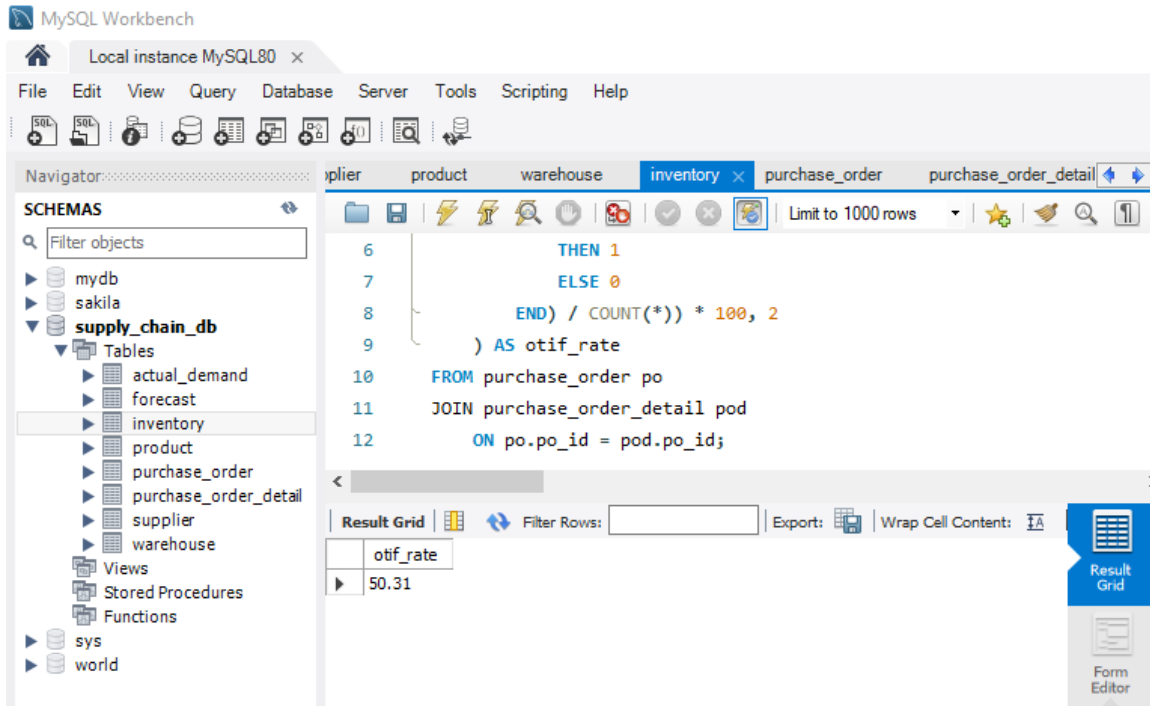


Figure 9. On-Time In Full

Due to dataset limitations, OTIF is approximated using delivery timeliness and order completeness indicators.

Since the dataset does not include a separate received quantity field, OTIF was approximated using delivery timeliness and order detail quantity completeness. The query evaluates whether an order line was delivered on time and had a valid ordered quantity.

8.3. Forecast Accuracy

This KPI measures how close the forecasted demand is to the actual demand.

Forecast Accuracy = 1 - (|Actual - Forecast| / Actual)

```

SELECT

    p.product_name,

    f.forecast_quantity,

    a.actual_quantity,

```

```

1 - ABS(a.actual_quantity - f.forecast_quantity) /
a.actual_quantity AS forecast_accuracy

```

```

FROM forecast f

```

```

JOIN actual_demand a

```

```

ON f.product_id = a.product_id

```

```

AND f.warehouse_id = a.warehouse_id

```

```

JOIN product p ON p.product_id = f.product_id;

```

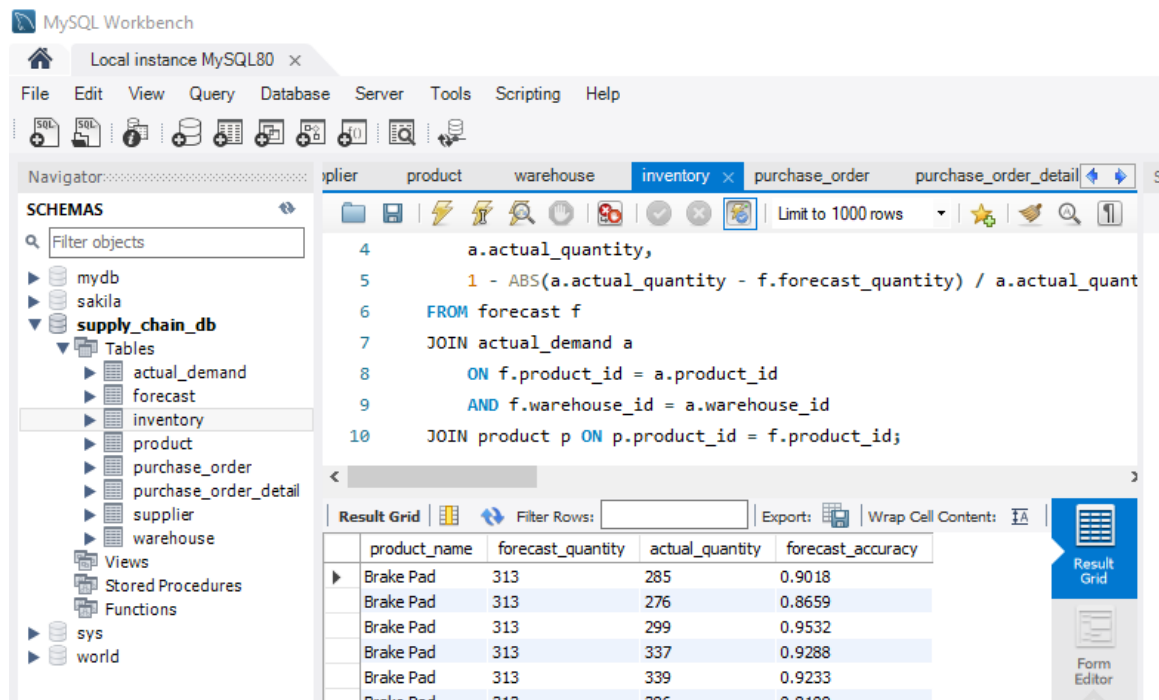


Figure 10. Forecast Accuracy

8.4. Inventory Turnover

This KPI measures how efficiently inventory is used over a period.

Inventory Turnover = Total Demand / Average Inventory

```

SELECT

```

```

  p.product_name,

```

```

  SUM(a.actual_quantity) / AVG(i.stock_quantity) AS
inventory_turnover

```

```

FROM actual_demand a

```

```

JOIN inventory i
    ON a.product_id = i.product_id
    AND a.warehouse_id = i.warehouse_id
JOIN product p ON p.product_id = a.product_id
GROUP BY p.product_name;

```

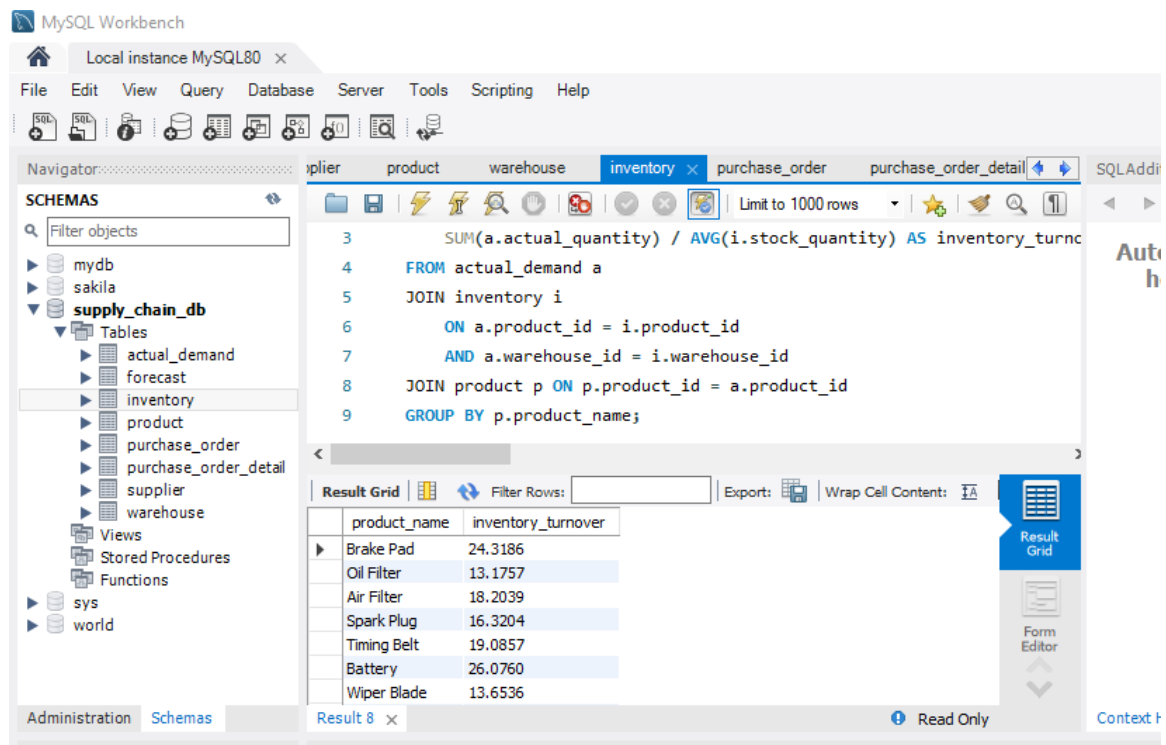


Figure 11. Inventory Turnover

Inventory turnover analysis at the product level reveals differences in inventory efficiency across products. For instance, products such as Battery and Brake Pad exhibit higher turnover rates, indicating faster inventory movement, while products like Oil Filter show relatively lower turnover, suggesting slower movement and potential overstock risk.

8.5. Stockout Rate

This KPI measures how often stock levels fall below safety stock.

Stockout Rate = (Number of Stockout Cases / Total Inventory Records) × 100

```

SELECT
    (SUM(CASE
        WHEN stock_quantity < safety_stock THEN 1

```

```
ELSE 0
```

```
END) / COUNT(*)) * 100 AS stockout_rate
```

```
FROM inventory;
```

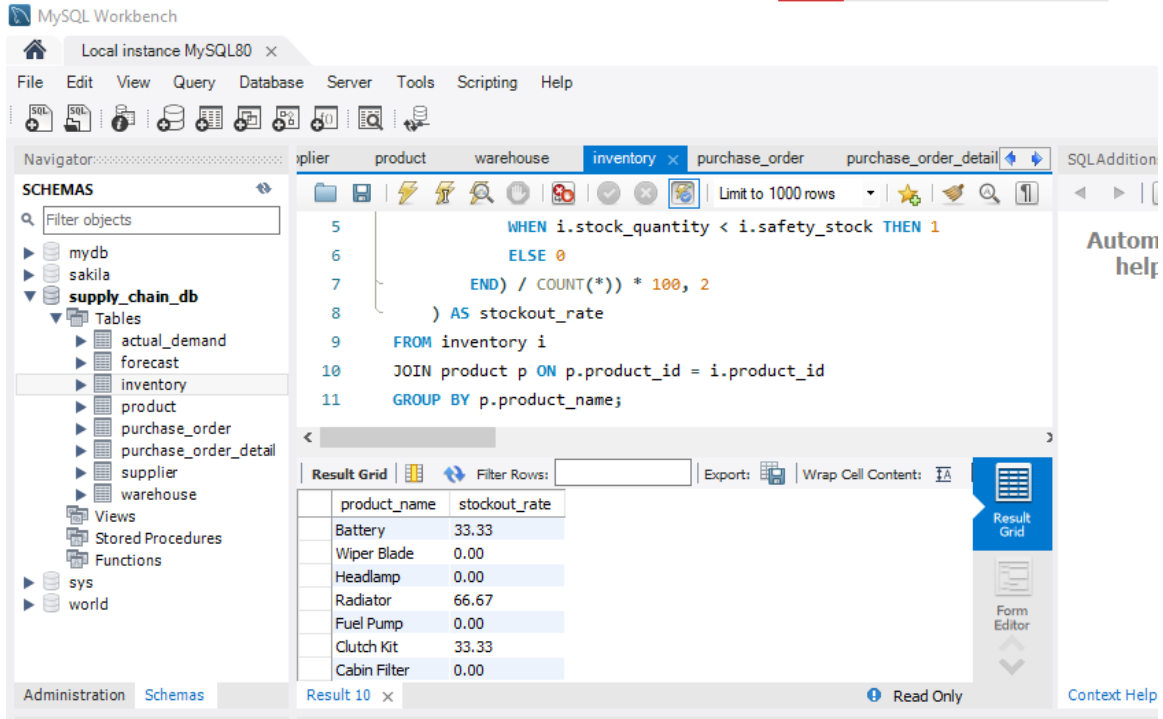


Figure 12. Stockout Rate

Product-level stockout analysis reveals that certain items, such as Radiator, experience significantly higher stockout rates compared to others. This indicates uneven inventory management and suggests that specific products require closer monitoring and improved replenishment strategies.

9. SQL Analysis

9.1. Supplier performance + delay analysis

```
SELECT
```

```
s.supplier_name,
```

```
COUNT(po.po_id) AS total_orders,
```

```
SUM(CASE
```

```
    WHEN po.actual_delivery_date > po.expected_delivery_date THEN 1
```

```
    ELSE 0
```

```

END) AS delayed_orders,

ROUND(

    SUM(CASE

        WHEN po.actual_delivery_date > po.expected_delivery_date

THEN 1

        ELSE 0

    END) / COUNT(po.po_id) * 100, 2

) AS delay_rate

FROM purchase_order po

JOIN supplier s ON po.supplier_id = s.supplier_id

GROUP BY s.supplier_name

ORDER BY delay_rate DESC;

```

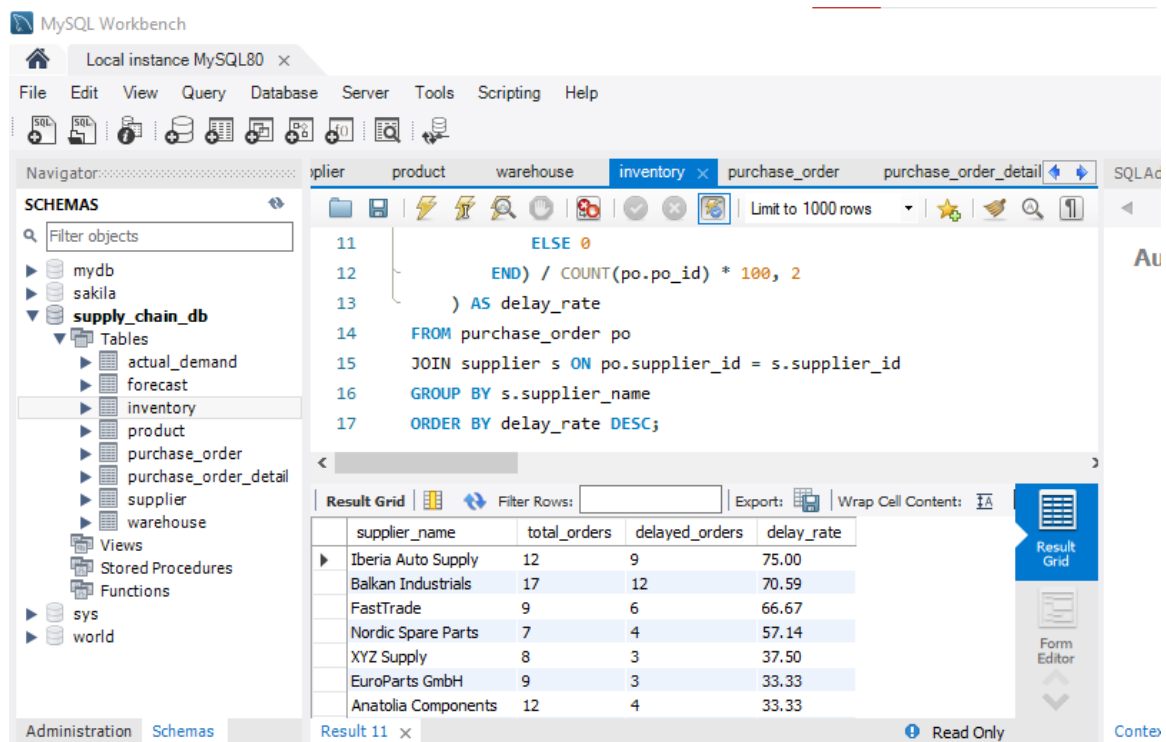


Figure 13. Supplier Performance + Delay Analysis

Supplier performance analysis reveals significant differences in delivery reliability. Some suppliers, such as Iberia Auto Supply and Balkan Industries, exhibit very high delay rates, exceeding 70%, indicating serious inefficiencies in delivery performance. In contrast,

suppliers like EuroParts GmbH and Anatolia Components show relatively better performance with lower delay rates.

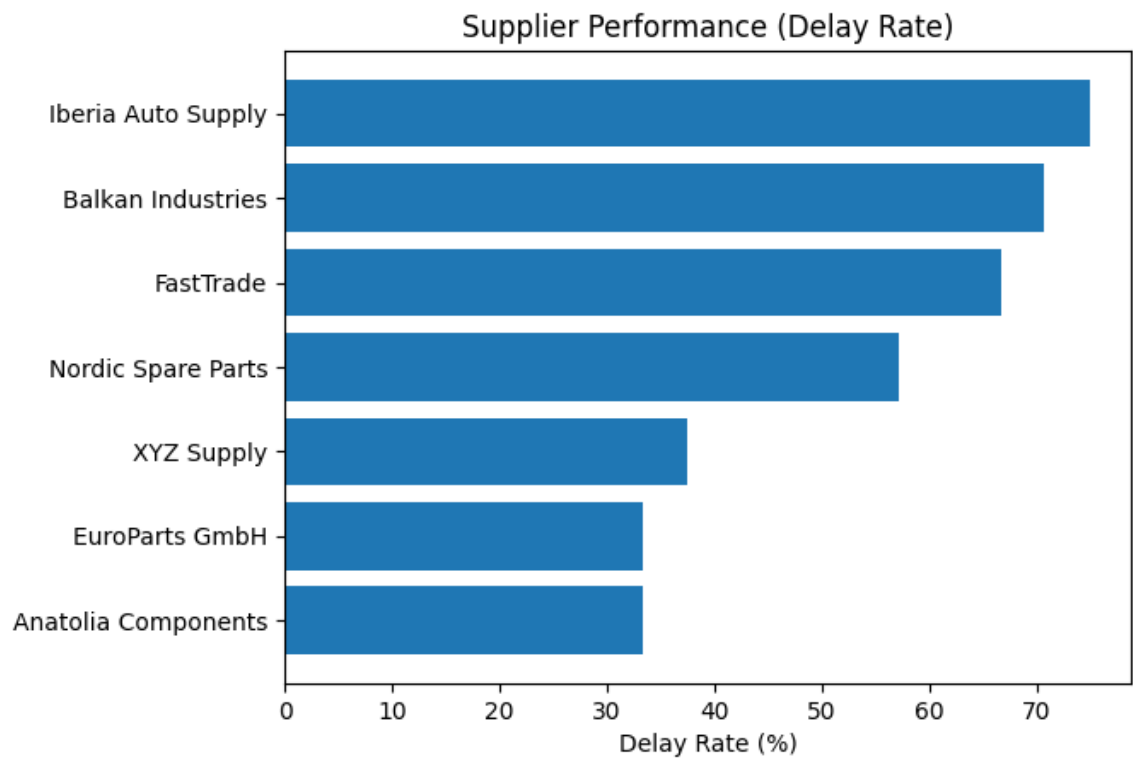


Figure 14. Supplier Performance + Delay Analysis Table

9.2. Demand trend analysis

```
SELECT
    demand_period,
    SUM(actual_quantity) AS total_actual_demand
FROM actual_demand
GROUP BY demand_period
ORDER BY demand_period;
```

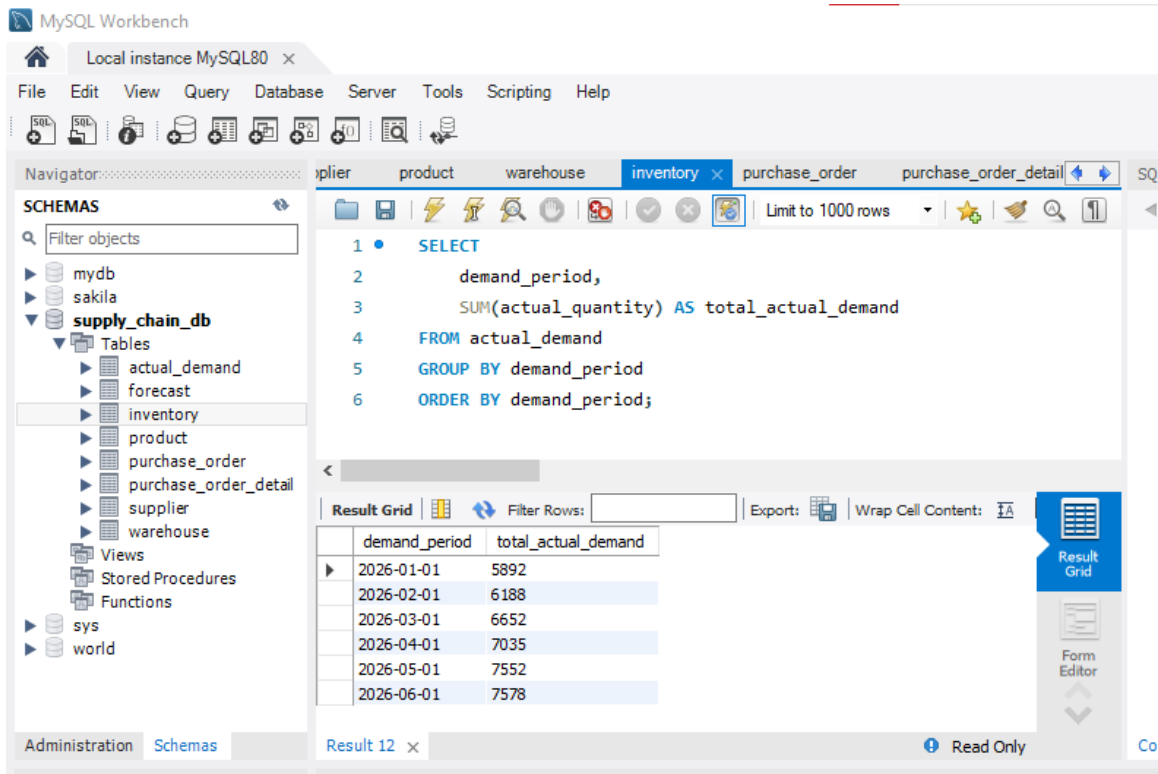


Figure 15. Demand Trend Analysis

Demand trend analysis shows a consistent increase in total demand over time. This upward trend indicates growing demand levels, which may require adjustments in inventory planning and supply chain capacity.

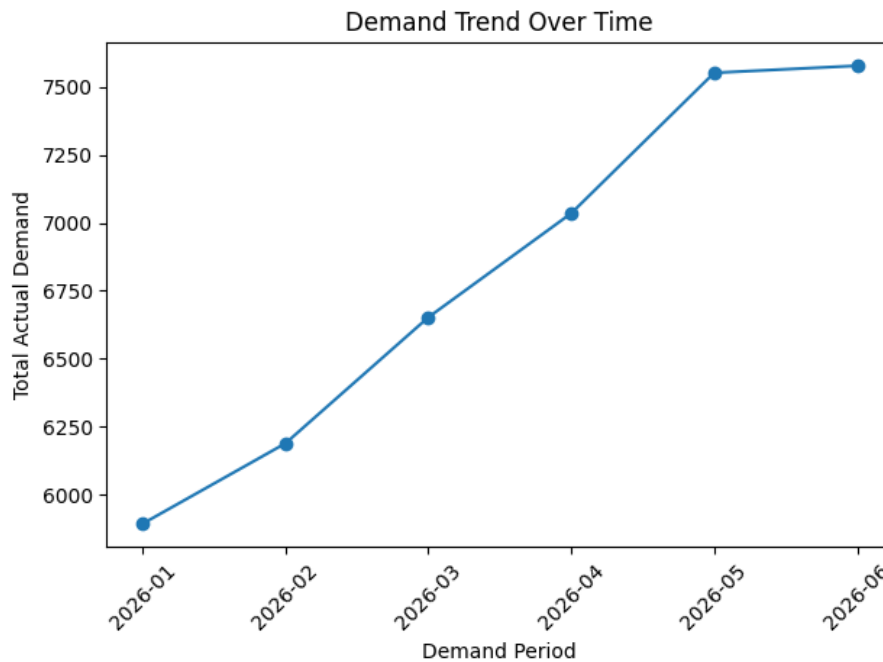


Figure 16. Demand Trend Analysis Graph

9.3. Forecast deviation analysis

```
SELECT

    p.product_name,

    w.warehouse_name,

    f.forecast_period,

    f.forecast_quantity,

    a.actual_quantity,

    (a.actual_quantity - f.forecast_quantity) AS forecast_deviation,

    ROUND(ABS(a.actual_quantity - f.forecast_quantity) /
a.actual_quantity * 100, 2) AS error_percentage

FROM forecast f

JOIN actual_demand a

    ON f.product_id = a.product_id

    AND f.warehouse_id = a.warehouse_id

    AND f.forecast_period = a.demand_period

JOIN product p ON f.product_id = p.product_id

JOIN warehouse w ON f.warehouse_id = w.warehouse_id

ORDER BY error_percentage DESC;
```

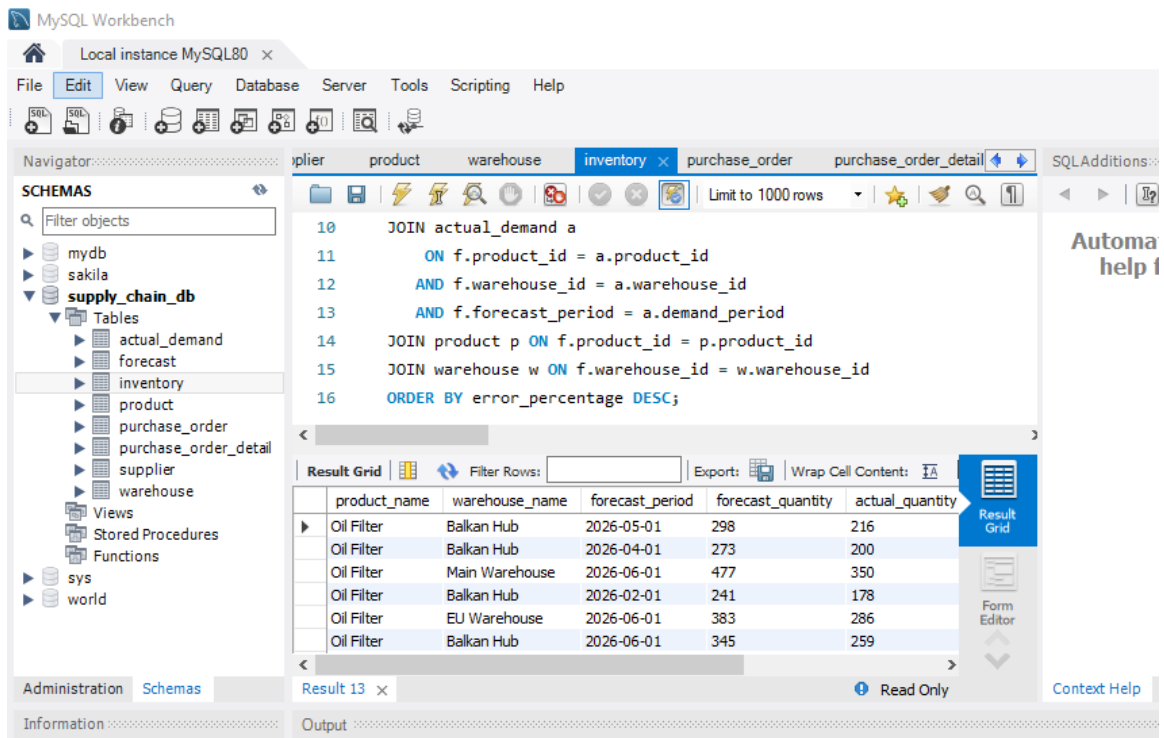


Figure 17. Forecast Deviation Analysis

Forecast deviation analysis reveals that forecasted demand is consistently higher than actual demand for several products and warehouses. This indicates a tendency toward overforecasting, which may lead to excess inventory and increased holding costs.

9.4. Window function

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with 'supply_chain_db' expanded, showing tables like 'inventory', 'product', and 'product_stockout'. The main editor shows a SQL query for 'Query 1' in the 'product' schema. The query uses a window function to rank products by stockout risk. The results are displayed in a table with columns 'product_name', 'stockout_rate', and 'risk_rank'.

```
12      GROUP BY p.product_name
13    )
14  SELECT
15    product_name,
16    stockout_rate,
17    RANK() OVER (ORDER BY stockout_rate DESC) AS risk_rank
18  FROM product_stockout;
```

product_name	stockout_rate	risk_rank
Radiator	66.67	1
Battery	33.33	2
Clutch Kit	33.33	2
Brake Pad	0.00	4
Oil Filter	0.00	4
Air Filter	0.00	4
Spark Plug	0.00	4

Figure 18. Stockout Risk Ranking

A CTE and window function were used to rank products based on stockout risk. The results help identify which products require closer inventory monitoring and improved replenishment planning.

A window function was used to rank products based on stockout risk. The analysis shows that Radiator has the highest risk level, followed by Battery and Clutch Kit. Several products exhibit no stockout risk, indicating stable inventory levels.

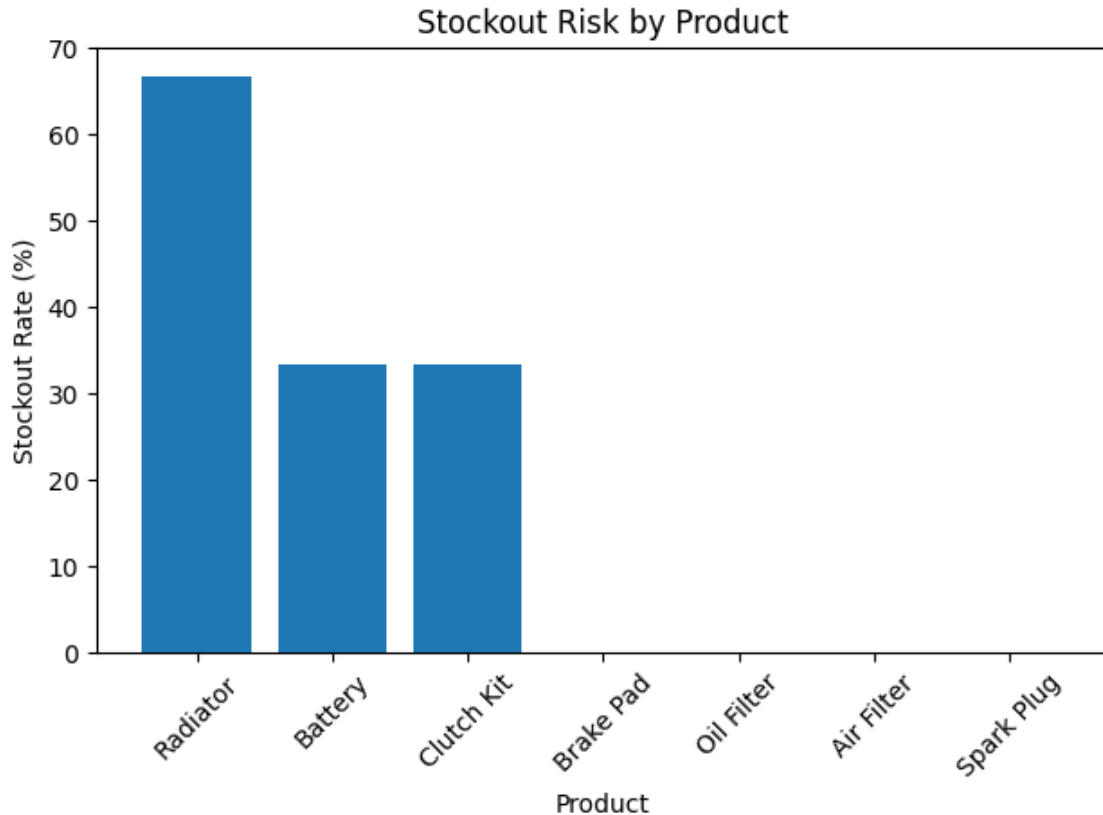


Figure 19. Stockout Risk Ranking Graph

Stockout Risk Classification by Product

The chart illustrates stockout risk levels across products. Radiator shows the highest risk with a significantly elevated stockout rate, while Battery and Clutch Kit fall into the medium-risk category. The remaining products exhibit no stockout risk, indicating stable inventory levels.

10. Views & Advanced SQL

To improve query reusability and simplify analytical processes, SQL views were created for frequently used analyses. The *vw_supplier_performance* view summarizes supplier delivery performance, while the *vw_forecast_accuracy* view provides a reusable structure for comparing forecasted and actual demand. These views enable more efficient querying and support modular data analysis.

10.1. Supplier Performance View

```
CREATE VIEW vw_supplier_performance AS
```

```
SELECT
```

```
    s.supplier_name,
```

```

COUNT(po.po_id) AS total_orders,
SUM(CASE
    WHEN po.actual_delivery_date > po.expected_delivery_date THEN 1
    ELSE 0
END) AS delayed_orders,
ROUND(
    SUM(CASE
        WHEN po.actual_delivery_date > po.expected_delivery_date
THEN 1
        ELSE 0
    END) / COUNT(po.po_id) * 100, 2
    ) AS delay_rate
FROM purchase_order po
JOIN supplier s ON po.supplier_id = s.supplier_id
GROUP BY s.supplier_name;

```

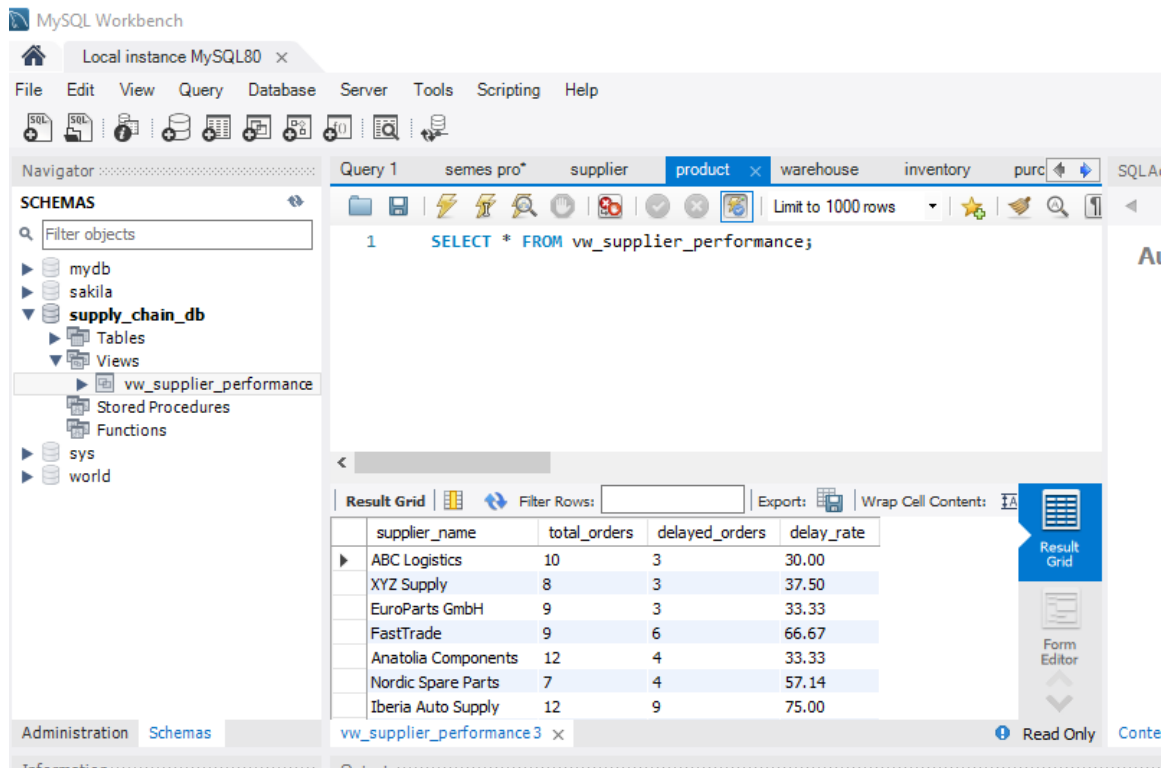


Figure 20. Supplier Performance View

10.2. Forecast Accuracy View

CREATE VIEW vw_forecast_accuracy AS

SELECT

p.product_name,

w.warehouse_name,

f.forecast_period,

f.forecast_quantity,

a.actual_quantity,

ROUND(

1 - ABS(a.actual_quantity - f.forecast_quantity) /
a.actual_quantity, 2

) AS forecast_accuracy

FROM forecast f

JOIN actual_demand a

```

ON f.product_id = a.product_id

AND f.warehouse_id = a.warehouse_id

AND f.forecast_period = a.demand_period

JOIN product p ON p.product_id = f.product_id

JOIN warehouse w ON w.warehouse_id = f.warehouse_id;

```

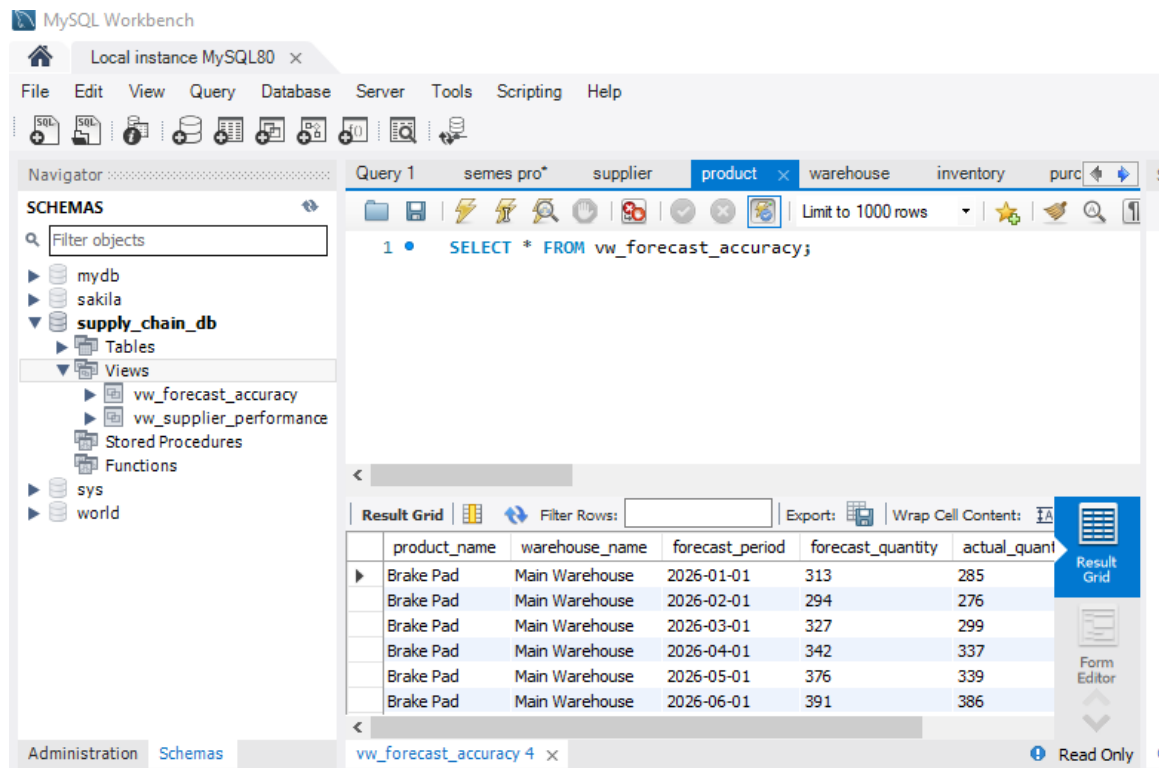


Figure 21. Forecast Accuracy View

11. Database Optimization

To improve query performance, indexing strategies were applied to frequently used columns such as `product_id`, `warehouse_id`, and `supplier_id`. These columns are commonly used in JOIN and filtering operations, making them critical for optimization. The use of indexes reduces query execution time and enhances overall database efficiency.

Indexes were already generated for foreign key columns during schema creation. Since these columns are frequently used in JOIN operations, they support query performance optimization.

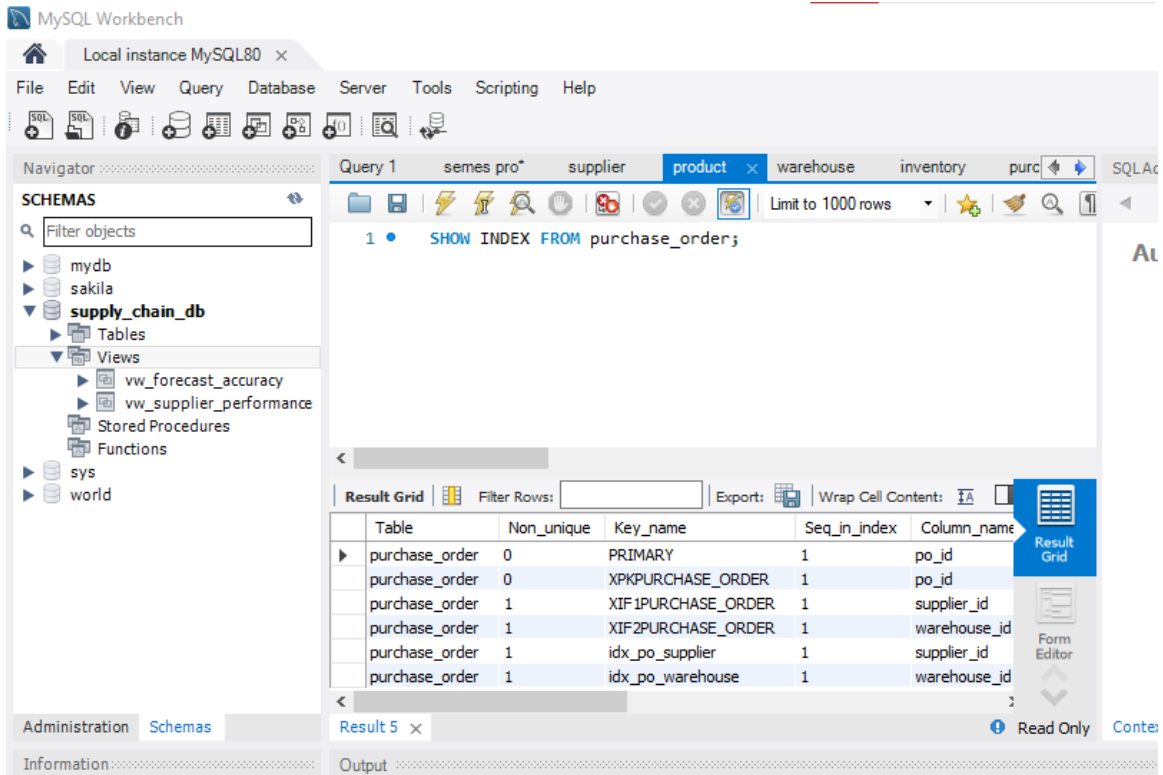


Figure 22. Show Index Command

Indexes on key columns such as `supplier_id` and `warehouse_id` were verified using the `SHOW INDEX` command. These indexes improve query performance by enabling faster data retrieval during JOIN operations.

12. Normalization

The database schema was designed according to normalization principles to eliminate redundancy and ensure data consistency.

First Normal Form (1NF) is achieved as all attributes contain atomic values with no repeating groups.

Second Normal Form (2NF) is satisfied since all non-key attributes are fully dependent on their respective primary keys.

Third Normal Form (3NF) is maintained by removing transitive dependencies, ensuring that each table contains only attributes directly related to its primary key.

As a result, the database structure supports efficient data storage, consistency, and maintainability.

13. Business Insights & Recommendations

13.1. Business Insights

The analysis of the supply chain dataset revealed several important insights regarding supplier performance, demand patterns, and inventory management.

13.1.1. Supplier Performance Issues

The supplier analysis showed that certain suppliers have significantly high delay rates. For example, Iberia Auto Supply and Balkan Industries exhibit delay rates above 70%, indicating serious inefficiencies in delivery performance. These delays can negatively impact overall supply chain reliability and service levels.

13.1.2. Increasing Demand Trend

Demand trend analysis indicates a steady increase in total demand over time. This consistent upward pattern suggests growing market demand, which requires proactive adjustments in supply chain capacity and inventory planning.

13.1.3. Forecast Accuracy Problems

Forecast deviation analysis revealed that forecasted demand is consistently higher than actual demand across multiple products and warehouses. This indicates a systematic overforecasting issue, which may lead to excess inventory, increased holding costs, and inefficient resource utilization.

13.1.4. Inventory Management Imbalance

Inventory turnover and stockout rate analyses showed that inventory performance varies significantly across products. While some products move quickly, others remain slower. Additionally, certain products such as Radiator exhibit high stockout risk, indicating insufficient inventory levels.

13.1.5. Uneven Risk Distribution

Stockout risk is not evenly distributed across products. Instead, it is concentrated in specific items, suggesting that a uniform inventory strategy may not be effective.

13.2. Recommendations

Based on the insights derived from the analysis, the following recommendations are proposed:

13.2.1. Improve Supplier Selection and Monitoring

High-risk suppliers should be closely monitored, and alternative suppliers should be considered. Implementing performance-based evaluation criteria can help improve delivery reliability.

13.2.2. Enhance Forecasting Models

The forecasting approach should be improved to reduce overestimation. Incorporating historical trends, seasonality, and real-time data can increase forecast accuracy.

13.2.3. Implement Product-Level Inventory Strategies

Inventory policies should be customized at the product level rather than applying a uniform approach. High-risk products should have higher safety stock levels and more frequent replenishment.

13.2.4. Align Inventory with Demand Trends

Given the increasing demand trend, inventory planning should be adjusted accordingly to prevent future stock shortages.

13.2.5. Reduce Excess Inventory

Overforecasting should be addressed to avoid unnecessary stock buildup and reduce holding costs.

Overall, the analysis demonstrates the importance of data-driven decision-making in supply chain management. By leveraging SQL-based analysis and performance metrics, organizations can improve efficiency, reduce risks, and enhance service levels.

14. Power BI Dashboard

14.1. On-Time Delivery Rate

On-Time Delivery Rate measures the percentage of orders delivered on or before the expected delivery date. It is a key performance indicator for evaluating supplier reliability and supply chain efficiency. A lower rate indicates delays in delivery, which may negatively impact customer satisfaction and operational performance.

14.2. Stockout Rate

Stockout Rate represents the proportion of inventory records where stock levels fall below the defined safety stock threshold. This metric helps identify potential inventory shortages and highlights weaknesses in inventory planning and replenishment strategies.

14.3. Inventory Turnover

Inventory turnover is expressed as a ratio rather than a percentage, and therefore may result in decimal values. The relatively high value observed is due to the limited and simulated nature of the dataset.

14.4. Forecast Accuracy

Forecast accuracy was calculated at the row level by comparing forecasted and actual demand, and then averaged across all records. This approach ensures that each observation contributes equally to the overall accuracy measure.

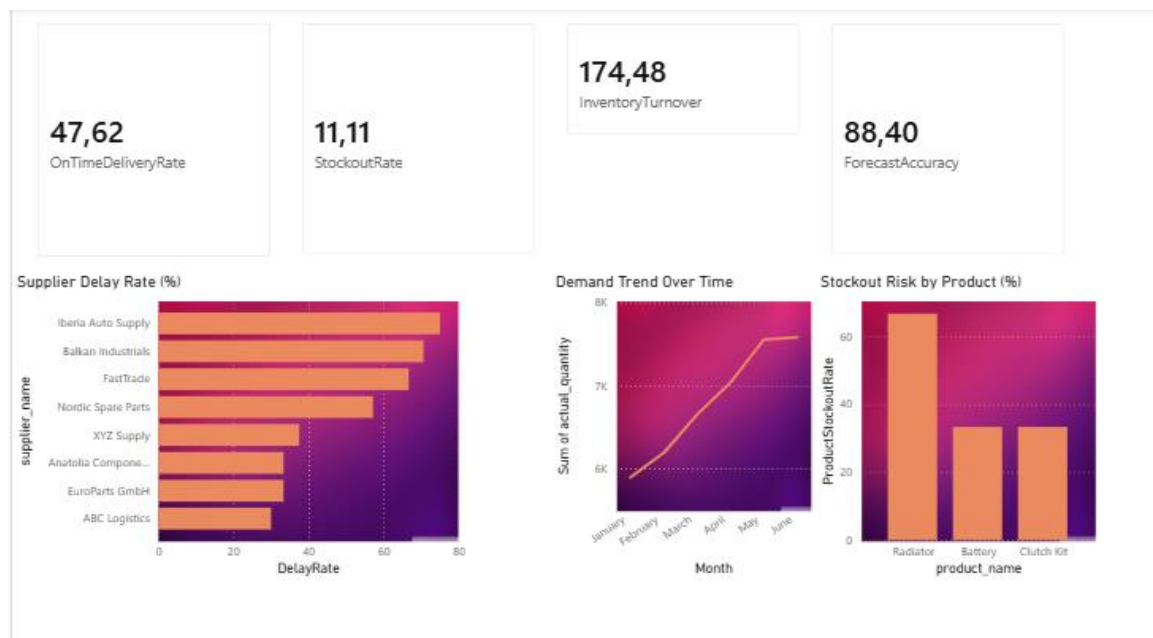


Figure 23. Power BI Dashboard

The dashboard provides an integrated view of supply chain performance through KPIs and visual analytics. Supplier delay rates, demand trends, and stockout risks can be monitored simultaneously, enabling faster and more informed operational decisions.

15. Analytical Thinking Approach

The analytical process followed a structured approach, starting with problem definition and data understanding. Data was first organized and modeled in a relational database, followed by cleaning and validation processes to ensure data quality.

SQL queries and KPI calculations were then used to analyze supplier performance, demand trends, forecasting accuracy, and inventory efficiency.

The insights derived from these analyses were used to identify key issues and support data-driven decision-making.

16. Limitations & Future Work

One of the main limitations of this project is the use of simulated data, which may not fully reflect real-world complexity. The dataset is limited in size and does not include external factors such as seasonality, market conditions, or unexpected disruptions.

For future work, real-time data integration and advanced forecasting methods such as machine learning models can be implemented to improve prediction accuracy and enhance decision-making capabilities.

17. Final Deliverables

The final deliverables of this project include:

- Entity-Relationship (ER) diagram
- SQL scripts for database creation and queries
- Sample dataset generated and processed in Excel
- Query outputs and KPI calculations
- Power BI dashboard with visualizations
- Final report summarizing the analysis and insights
- Flask-based front-end prototype for supply chain data entry and KPI monitoring

These deliverables collectively demonstrate the end-to-end development of a data-driven supply chain analysis project.

18. Supply Chain Management Front-End Prototype

A web-based front-end prototype was developed using Python Flask and MySQL to demonstrate user interaction with the supply chain database. The interface allows users to enter and manage supplier, product, purchase order, and forecast information through dedicated data entry modules.

In addition, a dashboard screen was implemented to display key performance indicators (KPIs), including On-Time Delivery Rate and Stockout Rate. The system provides a simple and user-friendly interface for monitoring supply chain performance and supporting operational decision-making.

The front-end prototype was implemented using Flask to establish communication between the user interface and the MySQL database. The following code snippet demonstrates a sample route for supplier data entry.

```

@app.route("/supplier", methods=["GET", "POST"])
def supplier():
    if request.method == "POST":
        conn = get_connection()
        cursor = conn.cursor()

        cursor.execute("""
            INSERT INTO supplier
            (supplier_id, supplier_name, country, status)
            VALUES (%s, %s, %s, %s)
            """, (
                request.form["supplier_id"],
                request.form["supplier_name"],
                request.form["country"],
                request.form["status"]
            ))
        conn.commit()
        conn.close()
        return redirect("/supplier")
    return render_template("supplier.html")

```

Example Flask route used for supplier data entry and database integration.

```

<form method="POST">
    <label>Supplier Name</label>
    <input type="text" name="supplier_name">
    <button type="submit">
        Add Supplier

```

</button>

</form>

Example HTML form used for user data entry.

18.1. Main Dashboard Screen

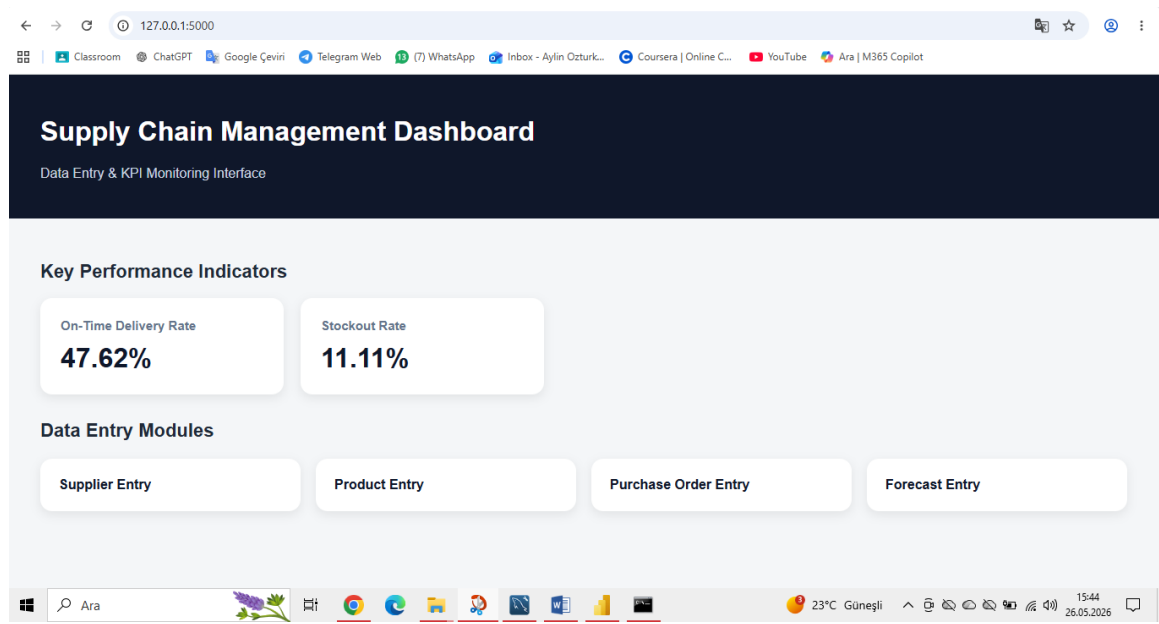


Figure 24. Main Dashboard Screen

18.2. Supplier Entry

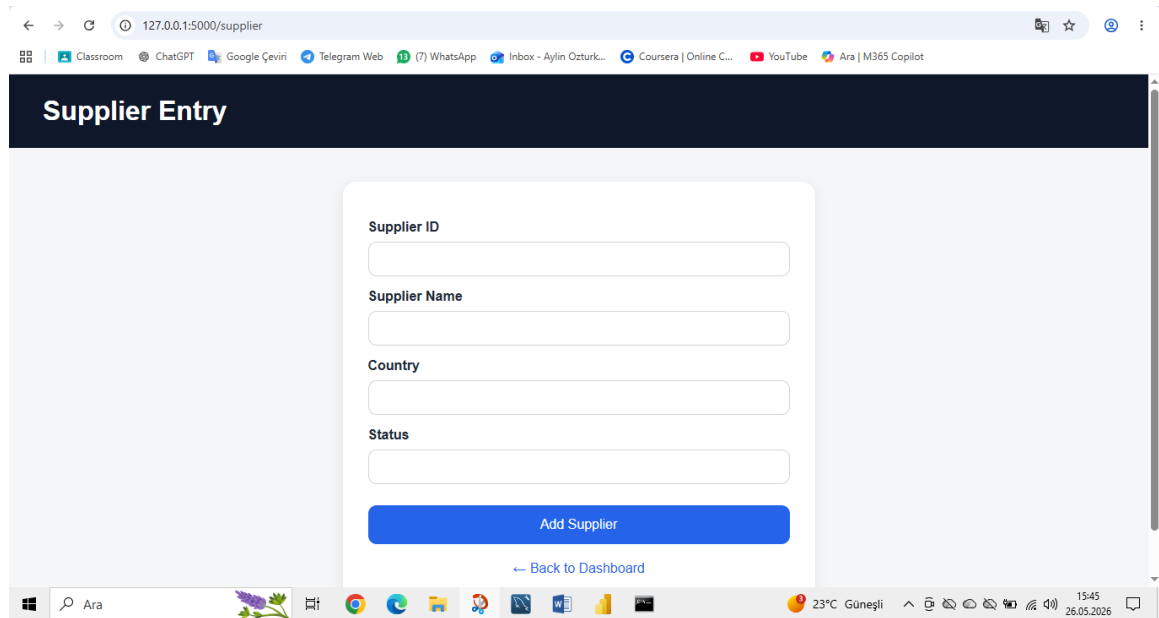


Figure 25. Supplier Entry

18.3. Product Entry

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/product". The browser's tab bar includes "Classroom", "ChatGPT", "Google Çeviri", "Telegram Web", "(7) WhatsApp", "Inbox - Aylin Ozturk...", "Coursera | Online C...", "YouTube", and "Ara | M365 Copilot". The page has a dark blue header with the text "Product Entry". Below the header is a light gray background with a white form containing the following fields: "Product ID", "Product Name", "Category", "Unit Price", and "Standard Lead Time". Each field is represented by a white rectangular input box. The Windows taskbar at the bottom shows the search bar with "Ara", several application icons, and system information including "23°C Güneşli", "15:46", and "26.05.2026".

Product Entry

Product ID

Product Name

Category

Unit Price

Standard Lead Time

Figure 26. Product Entry

18.4. Purchase Order Entry

The screenshot shows a web browser window with the address bar displaying "127.0.0.1:5000/purchase_order". The browser's tab bar includes "Classroom", "ChatGPT", "Google Çeviri", "Telegram Web", "(7) WhatsApp", "Inbox - Aylin Ozturk...", "Coursera | Online C...", "YouTube", and "Ara | M365 Copilot". The page has a dark blue header with the text "Purchase Order Entry". Below the header is a light gray background with a white form containing the following fields: "PO ID", "Supplier ID", "Warehouse ID", "Order Date", and "Expected Delivery Date". The "Order Date" and "Expected Delivery Date" fields include a date picker icon. The Windows taskbar at the bottom shows the search bar with "Ara", several application icons, and system information including "23°C Güneşli", "15:47", and "26.05.2026".

Purchase Order Entry

PO ID

Supplier ID

Warehouse ID

Order Date

Expected Delivery Date

Figure 27. Purchase Order Entry (1)

The screenshot shows a web application interface for entering a purchase order. The form is centered on a light gray background. It contains several input fields with labels: "Order Date", "Expected Delivery Date", "Actual Delivery Date", "Status", and "Total Amount". Each date field has a placeholder "gg . aa . yyyy" and a calendar icon. The "Status" field is a simple text input. Below these fields is a blue button labeled "Add Purchase Order" and a link labeled "← Back to Dashboard". The browser's taskbar at the bottom shows various icons and the system clock indicating 15:47 on 26.05.2026.

Order Date

gg . aa . yyyy

Expected Delivery Date

gg . aa . yyyy

Actual Delivery Date

gg . aa . yyyy

Status

Total Amount

Add Purchase Order

← Back to Dashboard

Figure 28. Purchase Order Entry (2)

18.5. Forecast Entry

The screenshot shows a web application interface for entering a forecast. The page has a dark blue header with the title "Forecast Entry". Below the header is a form with several input fields with labels: "Forecast ID", "Product ID", "Warehouse ID", "Forecast Period", and "Forecast Quantity". The "Forecast Period" field has a placeholder "gg . aa . yyyy" and a calendar icon. The browser's taskbar at the bottom shows various icons and the system clock indicating 15:47 on 26.05.2026.

Forecast Entry

Forecast ID

Product ID

Warehouse ID

Forecast Period

gg . aa . yyyy

Forecast Quantity

Figure 29. Forecast Entry (1)

Product ID

Warehouse ID

Forecast Period

gg . aa . yyyy

Forecast Quantity

Created Date

gg . aa . yyyy

Add Forecast

[← Back to Dashboard](#)

Ara



23°C Güneşli

15:48
26.05.2026

Figure 30. Forecast Entry (2)

19. Conclusion

This project demonstrated how SQL, data modeling, Power BI, and front-end technologies can be integrated to analyze supply chain performance. Through KPI calculations, supplier analysis, forecast evaluation, and inventory monitoring, meaningful business insights were generated to support data-driven decision-making.

The project also highlighted the practical application of database management concepts, including normalization, ETL, query optimization, and analytical reporting. Overall, the study provides a foundation for scalable and analytics-driven supply chain management systems.

20. References

Chopra, S., & Meindl, P. (2016). Supply Chain Management: Strategy, Planning, and Operation (6th ed.). Pearson.

Christopher, M. (2016). Logistics & Supply Chain Management (5th ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

Oracle. (2026). MySQL 8.0 Reference Manual. Oracle. Retrieved from <https://dev.mysql.com/doc/refman/8.0/en/>

Microsoft. (2026). Power BI documentation. Microsoft Learn. Retrieved from <https://learn.microsoft.com/en-us/power-bi/>

Pallets Projects. (2026). Flask Documentation. Retrieved from <https://flask.palletsprojects.com/>

Microsoft. (2026). Microsoft Excel documentation. Microsoft Support. Retrieved from <https://support.microsoft.com/excel>